

CPSC-406 Report

Sami Hammoud
Chapman University

March 7, 2026

Abstract

Contents

1	Introduction	1
2	Week by Week	1
2.1	Week 1	1
2.2	Week 2: Operations on Automata	4
2.3	Week 3	8
2.4	Week 4	11
3	Synthesis	12
4	Evidence of Participation	12
5	Conclusion	12

1 Introduction

2 Week by Week

2.1 Week 1

Exercise 1

Consider the following DFAs $\mathcal{A}_1$ and $\mathcal{A}_2$. Complete the following table with all 8 possible strings of length 3 over $\{a, b\}$ (columns: Accepted by $\mathcal{A}_1$? and Accepted by $\mathcal{A}_2$?).

Answer to Exercise 1.1. Here is my completed table (words of length 3 over $\{a, b\}$):

word	Accepted by $\mathcal{A}_1$?	Accepted by $\mathcal{A}_2$?
<i>aaa</i>	No	Yes
<i>aab</i>	Yes	No
<i>aba</i>	No	No
<i>abb</i>	No	No
<i>baa</i>	No	Yes
<i>bab</i>	No	No
<i>bba</i>	No	No
<i>bbb</i>	No	No

Exercise 1.2

Describe the languages $L(\mathcal{A}_k)$ accepted by $\mathcal{A}_k$ for $k = 1, 2$.

Answer to Exercise 1.2. For $k = 1$, $L(\mathcal{A}_1)$ is just the single word *aab*. For $k = 2$, $L(\mathcal{A}_2)$ is all words over $\{a, b\}$ that end in *at least two a's in a row* (so the last two symbols are *aa*).

Exercise 2 (Designing DFAs)

Design DFAs whose accepted languages are given as follows:

1. All the words that end with *ab*
2. All the words that contain *aba*
3. All the words that contain an odd number of *a*'s and an odd number of *b*'s
4. All the words that contain an even number of *a*'s and an odd number of *b*'s
5. All the words such that any three consecutive characters contain at least one *a*
6. All the words that contain *bbb*

What do you notice when comparing the various automata?

Answers for Exercise 2.

(1) **Words that end with *ab*.** I use three states that remember the last one or two symbols. The DFA is:

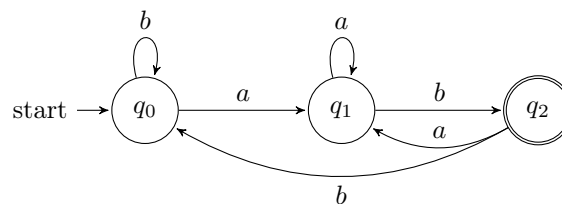


Figure 1: DFA for all words that end with *ab*.

(2) **Words that contain *aba*.** Here I track how much of the pattern *aba* I have just seen as a suffix.

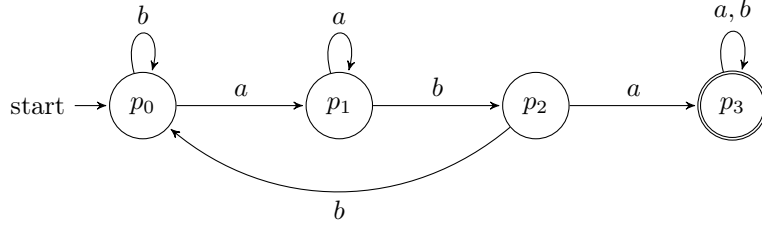


Figure 2: DFA for all words that contain aba as a substring.

(3) Odd number of a 's and odd number of b 's. For this and the next language I use the same 4-state parity automaton, but choose different accepting states. The four states remember the parity (even/odd) of the number of a 's and b 's seen so far.

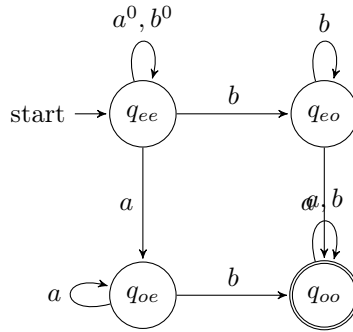


Figure 3: Parity DFA: state q_{xy} means x -parity for a 's and y -parity for b 's. For language (3) we accept q_{oo} (odd a 's and odd b 's).

(4) Even number of a 's and odd number of b 's. Using the same parity DFA in Figure 3, this time the accepting state is q_{eo} (even a 's, odd b 's) instead of q_{oo} .

(5) Any three consecutive characters contain at least one a . This is the same as saying that we never see bbb as a substring. I track runs of consecutive b 's up to length 3; hitting three b 's in a row moves to a dead (rejecting) state.

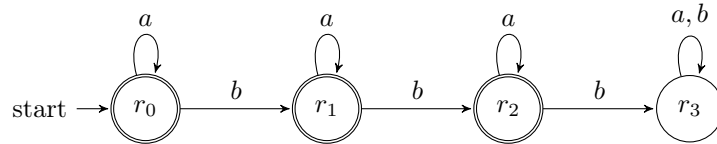


Figure 4: DFA that forbids bbb ; states r_0, r_1, r_2 are accepting, r_3 is a dead state reached after three b 's in a row.

(6) Words that contain bbb . I reuse the same transition structure as in Figure 4, but now r_3 is the *only* accepting state (once we have seen three consecutive b 's we stay in an accepting sink).

What I notice. Several of these automata reuse the same underlying structure but just change which states are accepting (for example (3) vs. (4), and (5) vs. (6)). In other words, a lot of the “different”

languages here really come from the same core DFA with a different choice of final states.

Question

How might you combine the languages $L(\mathcal{A}_1)$ and $L(\mathcal{A}_2)$ from Exercise 1 using set operations (union, intersection, complement)? Could you represent the logic with boolean logic exclusively?

2.2 Week 2: Operations on Automata

Product automata

Exercise 1 (Product automata)

Consider the following automata $\mathcal{A}^{(1)}$ and $\mathcal{A}^{(2)}$.

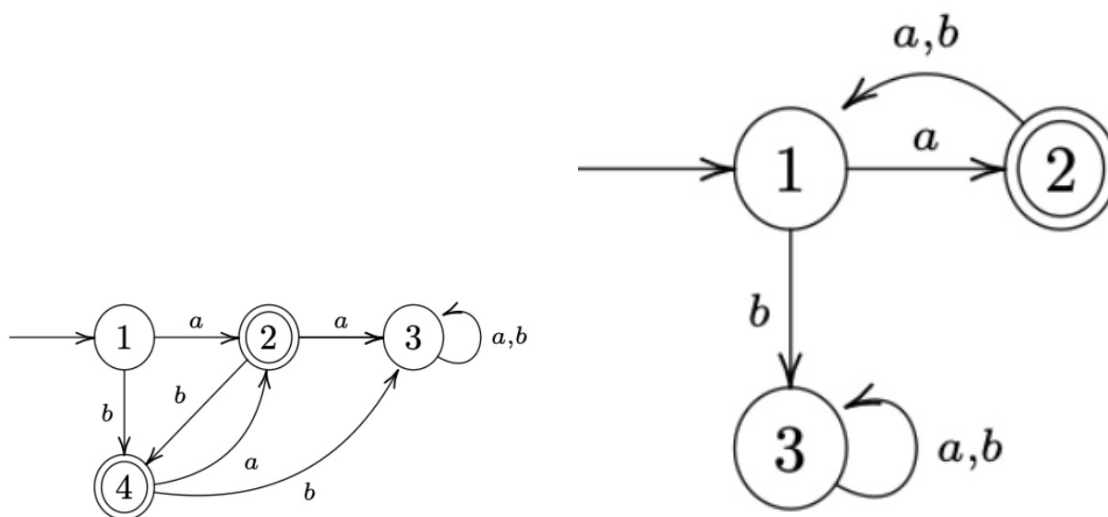


Figure 5: $\mathcal{A}^{(1)}$ (left) and $\mathcal{A}^{(2)}$ (right).

1. Describe $L(\mathcal{A}^{(1)})$ and $L(\mathcal{A}^{(2)})$.

Answer:

- $L(\mathcal{A}^{(1)})$: Words include $a, b, ba, bab, baba, ab, aba, abab$, etc. I can see that each character must alternate based on its successive character.
- $L(\mathcal{A}^{(2)})$: Words include a, axa (where x doesn't matter). I see the word must start with a , and must add to its word length in increments of 2, every other character being a .

2. Construct the intersection automaton $\mathcal{A}$ for $\mathcal{A}^{(1)}$ and $\mathcal{A}^{(2)}$ by drawing its transition diagram (omit unreachable states).

Answer: Below is the intersection automaton diagram.

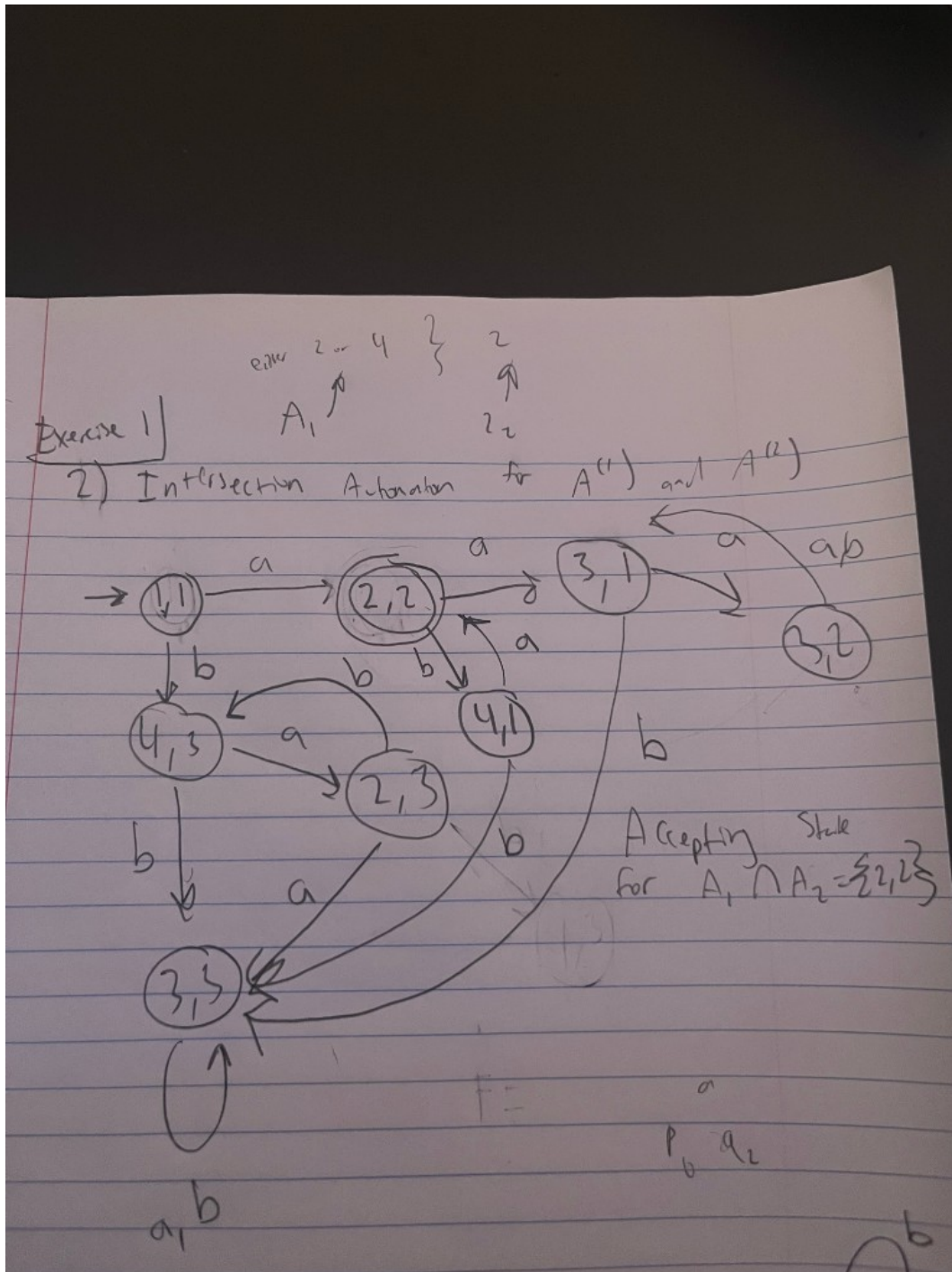


Figure 6: Intersection automaton $\mathcal{A}$ for $\mathcal{A}^{(1)}$ and $\mathcal{A}^{(2)}$ (omit unreachable states).

Note: $\{2, 2\}$ as accepting state for intersection product automaton.

3. Argue that $L(\mathcal{A}) = L(\mathcal{A}^{(1)}) \cap L(\mathcal{A}^{(2)})$.

Answer: Since $L(\mathcal{A}^{(1)})$ includes words such as a , b , ba , bab , $baba$ and ab , aba , $abab$, and $L(\mathcal{A}^{(2)})$ includes words that (1) must start with a —so we can disregard the words that start with b in $L(\mathcal{A}^{(1)})$, leaving us with a , ab , aba , etc.—and (2) the requirement for $L(\mathcal{A}^{(2)})$ is to be in increments of two, we are left with patterns: a , aba , $ababa$, etc. Thus we obtain the automaton of both intersection.

4. **How can you change $\mathcal{A}$ to get an automaton $\mathcal{A}'$ with $L(\mathcal{A}') = L(\mathcal{A}^{(1)}) \cup L(\mathcal{A}^{(2)})$?**

Answer: In the product automaton we would add the accepting states to not be exclusive to both, but inclusive of either one of the subset DFA's accepting state.

Exercise 2 (More automata)

Consider the following automata $\mathcal{B}^{(1)}$ and $\mathcal{B}^{(2)}$.

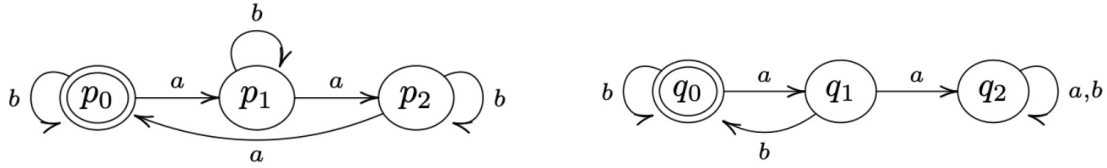


Figure 7: $\mathcal{B}^{(1)}$ (left) and $\mathcal{B}^{(2)}$ (right).

(Recall: the complement of a language $L \subseteq \Sigma^*$ is $\bar{L} := \Sigma^* \setminus L$.)

1. **Describe $L(\mathcal{B}^{(1)})$ and $L(\mathcal{B}^{(2)})$.**

Answer:

Figure 8: $L(\mathcal{B}^{(1)})$ rules: include 3 a 's at a time, any amount of b 's. $L(\mathcal{B}^{(2)})$ rules: b must follow an a , any amount of b 's. DFA diagram for language 2.

2. **Construct the intersection automaton $\mathcal{B}$ for $\mathcal{B}^{(1)}$ and $\mathcal{B}^{(2)}$ (omit unreachable states).**

Answer: Below is the intersection automaton diagram.

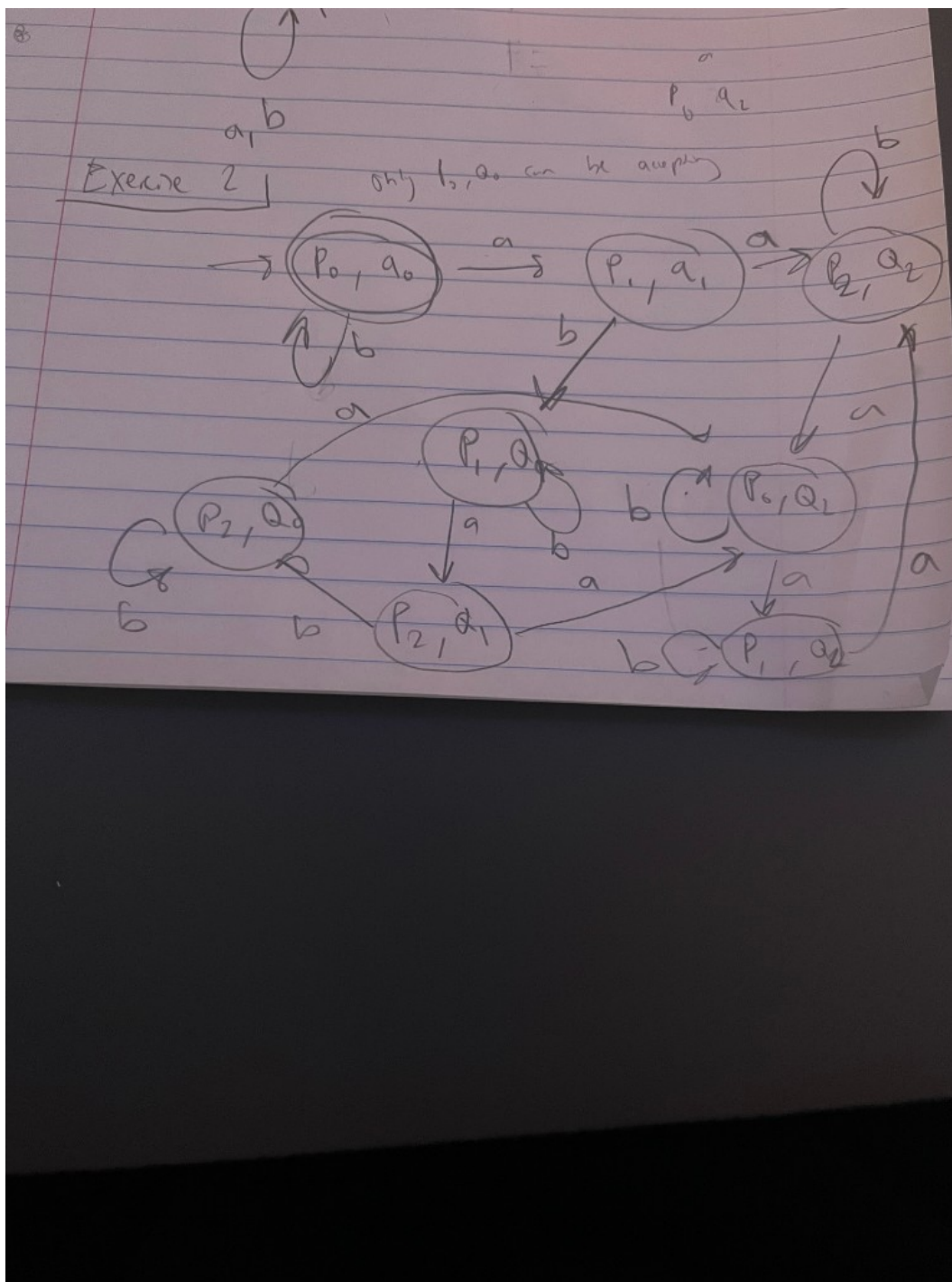


Figure 9: Intersection automaton $\mathcal{B}$ for $\mathcal{B}^{(1)}$ and $\mathcal{B}^{(2)}$ (omit unreachable states).

Note: (P_0, Q_0) as accepting state.

3. Argue that $L(\mathcal{B}) = L(\mathcal{B}^{(1)}) \cap L(\mathcal{B}^{(2)})$.

Answer: Both automata have the rule that any amount of b 's is allowed. But we must combine the rules regarding a 's. $L(\mathcal{B}^{(1)})$ requires 3 a 's to be included at a time per word, while $\mathcal{B}^{(2)}$ requires the rule that a must be followed by a b . This gives us the conclusion that words that do include an a must come in pairs of 3 a 's with b following the last a .

4. **How can you change $\mathcal{B}$ to get an automaton $\mathcal{B}'$ with $L(\mathcal{B}') = L(\mathcal{B}^{(1)}) \cup \overline{L(\mathcal{B}^{(2)})}$?**

Answer: The Union is equivalent to its intersection, no possible change.

Exercise 3

Let $\mathcal{A}$ be a DFA and q a state of $\mathcal{A}$ such that from q , every input symbol a leaves us in q (i.e. $\delta(q, a) = q$ for all a). Show by induction on the length of the input that for any input string w , we still end up in q (i.e. $\delta^*(q, w) = q$).

Answer: We show it by induction on the length of w . (Here $\delta^*(q, w)$ is just the state we end up in after starting at q and reading the whole string w .)

When the input has length 0, the string is empty. So we don't read anything and we stay at q . So $\delta^*(q, w) = q$ in that case.

Now suppose we've already shown that for every string of length n , we end up back at q . Take a string w of length $n + 1$. We can write it as $w = va$ (some string v of length n plus one symbol a). After reading v we're in $\delta^*(q, v)$, and by our assumption that's q . Then we read a , and we're in $\delta(q, a)$, which is q by the given rule. So after reading w we're still in q , i.e. $\delta^*(q, w) = q$.

So by induction, for all input strings w we have $\delta^*(q, w) = q$.

Question

If you build the product automaton for $\mathcal{B}^{(1)}$ and $\mathcal{B}^{(2)}$ but change the accepting set to get the *union* $L(\mathcal{B}^{(1)}) \cup L(\mathcal{B}^{(2)})$ instead of the intersection, how many states must be accepting?

2.3 Week 3

Homework 1 (NFA)

1. Describe the language accepted by $\mathcal{A}$ using a regular expression.
2. Specify the NFA $\mathcal{A}$ in the form $(Q, \Sigma, \delta, q_0, F)$.
3. Find all paths in $\mathcal{A}$ for the word $w = abab$; represent in a diagram.
4. Construct the determinization $\mathcal{A}^D$ of $\mathcal{A}$ using the power set construction. Create both a transition table and a graphical depiction of $\mathcal{A}^D$.

Answer (NFA specification). Here is how I write the NFA $\mathcal{A}$ formally:

$$Q = \{0, 1, 2, 3\}, \quad \Sigma = \{a, b, c\}, \quad q_0 = 0, \quad F = \{2, 3\}.$$

For the transition function $\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$ I use

$$\begin{aligned} \delta(0, a) &= \{0, 1\}, & \delta(0, b) &= \{0\}, & \delta(0, c) &= \emptyset, \\ \delta(1, b) &= \{2\}, & \delta(1, c) &= \{2\}, \end{aligned}$$

and any transition not listed goes to the empty set (i.e. there is no move).

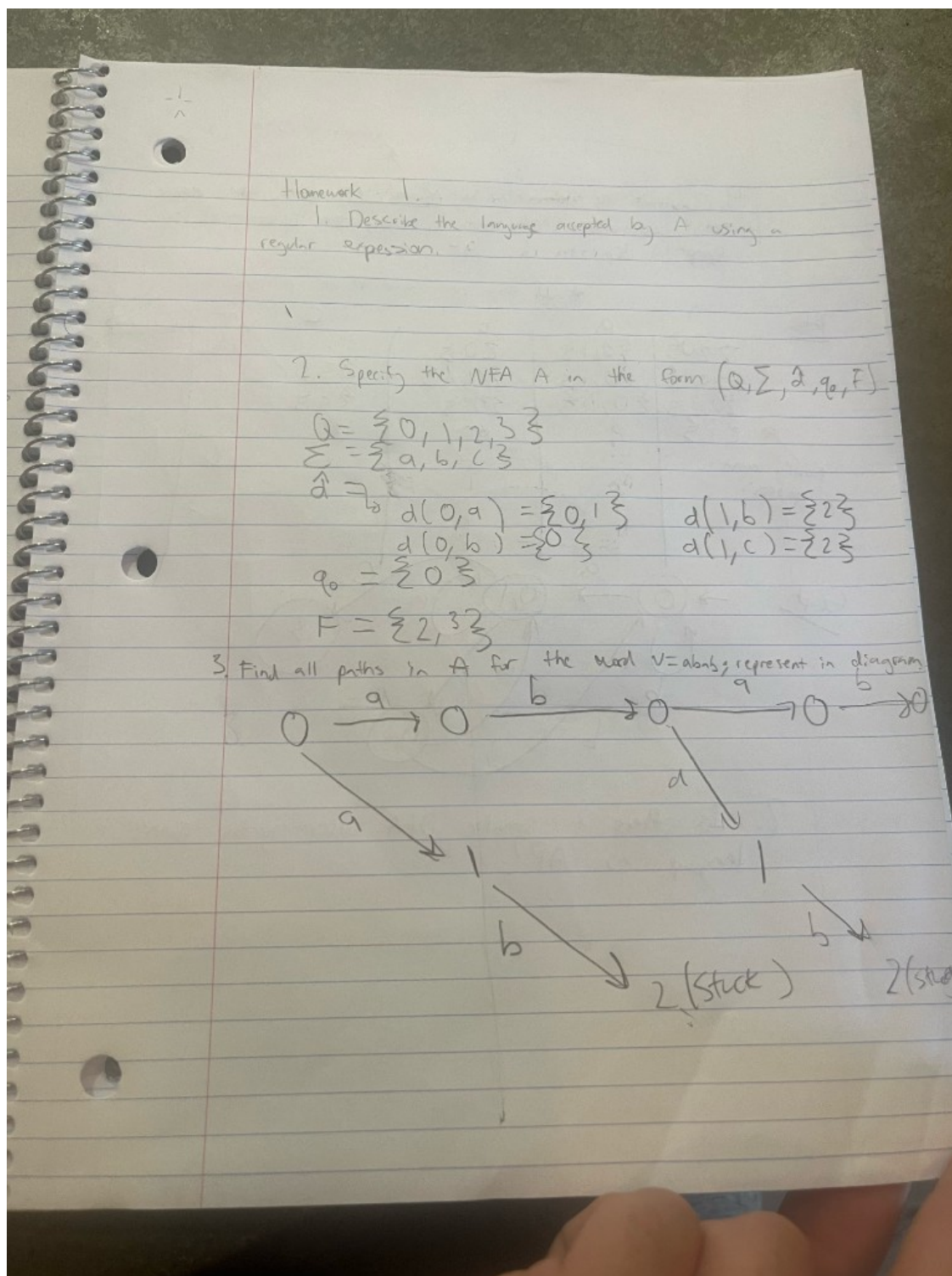


Figure 10: Homework 1: NFA specification $(Q = \{0, 1, 2, 3\}, \Sigma = \{a, b, c\}, q_0 = 0, F = \{2, 3\})$ and all paths for $w = abab$.

Transition table for $\mathcal{A}^D$ over $\{a, b, c\}$. From my construction I focus on the following subset-states:

$$\{0\}, \{0, 1\}, \{0, 2\}, \{3\}.$$

The transitions on a , b , and c are:

State S	on input a	on input b	on input c
$\{0\}$	$\{0, 1\}$	$\{0\}$	$\emptyset$
$\{0, 1\}$	$\{0, 1\}$	$\{0, 2\}$	$\{3\}$
$\{0, 2\}$	$\{0, 1\}$	$\{0\}$	$\emptyset$
$\{3\}$	$\emptyset$	$\emptyset$	$\emptyset$

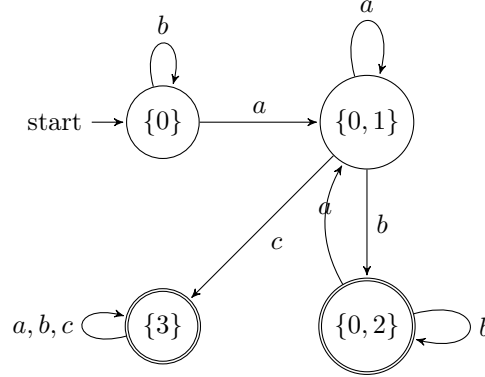


Figure 11: Determinized DFA $\mathcal{A}^D$ for Homework 2 over alphabet $\{a, b, c\}$. Accepting states are the subsets that contain an original accepting state ($\{0, 2\}$ and $\{3\}$).

Homework 2 (DFA and minimization)

Determine $\mathcal{A}^D$ of $\mathcal{A}$ using the power set construction, now with alphabet $\{0, 1\}$. Create both a transition table and a graphical depiction of $\mathcal{A}^D$. Then decide whether there can be a smaller DFA accepting the same language.

Transition table over $\{0, 1\}$. From the NFA given in the homework, I use the following subset-states:

$$q_0, \{q_0, q_1\}, \{q_0, q_1, q_2\}, \{q_0, q_3\}, \{q_0, q_1, q_3\}, \{q_0, q_1, q_2, q_3\}.$$

The determinized DFA $\mathcal{A}^D$ then has this transition table:

State S	on input 0	on input 1
q_0	q_0	$\{q_0, q_1\}$
$\{q_0, q_1\}$	q_0	$\{q_0, q_1, q_2\}$
$\{q_0, q_1, q_2\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_2\}$
$\{q_0, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_3\}$
$\{q_0, q_1, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_2, q_3\}$
$\{q_0, q_1, q_2, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_2, q_3\}$

Any subset that contains the original accepting state(s) is accepting in $\mathcal{A}^D$.

Smaller DFA? Yes. In this table the last two rows (for $\{q_0, q_1, q_2, q_3\}$ and for $\{q_0, q_3\}$) have exactly the same outgoing transitions and the same accepting/non-accepting status. That means these two states are equivalent and can be merged into a single state, giving a smaller DFA that still recognizes the same language.

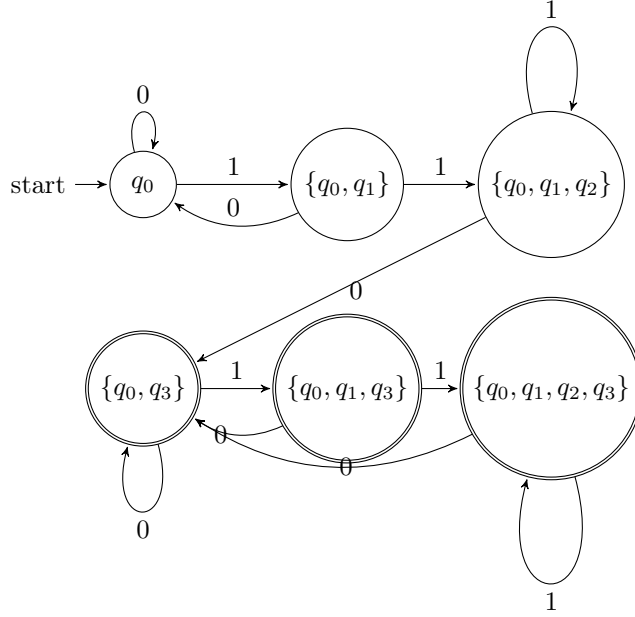


Figure 12: Determinized DFA $\mathcal{A}^D$ for Homework 2 over alphabet $\{0,1\}$. Accepting states are the subsets that contain the original accepting state q_3 .

Question

After building $\mathcal{A}^D$ from an NFA $\mathcal{A}$ using the power set construction, when can we get a smaller DFA that still accepts the same language?

2.4 Week 4

Consider the following DFA $\mathcal{A}$: there is a state p (initial and final) with transition a to itself and transition b to state q ; state q has transition b to itself and transition a to p .

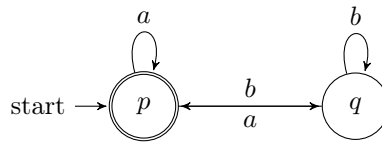


Figure 13: DFA $\mathcal{A}$ for Week 4: p is initial and final; $p \xrightarrow{a} p$, $p \xrightarrow{b} q$, $q \xrightarrow{b} q$, $q \xrightarrow{a} p$.

Question. For $p = 1$ and $q = 2$, compute via Kleene's algorithm all regular expressions $R_{ij}^{(k)}$ and from this a resulting regular expression R describing $L(\mathcal{A})$, i.e., such that $L(R) = L(\mathcal{A})$.

Answer. I label the states so that state 1 is p and state 2 is q . In Kleene's algorithm, $R_{ij}^{(k)}$ denotes the set of all strings that label paths from i to j with intermediate states (if any) in $\{1, \dots, k\}$. The recurrence is

$$R_{ij}^{(k)} = R_{ij}^{(k-1)} \cup R_{ik}^{(k-1)} (R_{kk}^{(k-1)})^* R_{kj}^{(k-1)}.$$

I write union as $+$ in expressions below.

Base case $k = 0$ (no intermediate states; only direct edges and ε on same state):

$$R_{11}^{(0)} = \varepsilon + a, \quad R_{12}^{(0)} = b, \quad R_{21}^{(0)} = a, \quad R_{22}^{(0)} = \varepsilon + b.$$

Inductive step $k = 1$ (intermediate state 1 allowed). Using $(R_{11}^{(0)})^* = (\varepsilon + a)^* = a^*$:

$$\begin{aligned} R_{11}^{(1)} &= R_{11}^{(0)} + R_{11}^{(0)} (R_{11}^{(0)})^* R_{11}^{(0)} = (\varepsilon + a) + (\varepsilon + a) a^* (\varepsilon + a) = a^*; \\ R_{12}^{(1)} &= R_{12}^{(0)} + R_{11}^{(0)} (R_{11}^{(0)})^* R_{12}^{(0)} = b + (\varepsilon + a) a^* b = a^* b; \\ R_{21}^{(1)} &= R_{21}^{(0)} + R_{21}^{(0)} (R_{11}^{(0)})^* R_{11}^{(0)} = a + a a^* (\varepsilon + a) = a a^*; \\ R_{22}^{(1)} &= R_{22}^{(0)} + R_{21}^{(0)} (R_{11}^{(0)})^* R_{12}^{(0)} = (\varepsilon + b) + a a^* b = (\varepsilon + b) + a^+ b. \end{aligned}$$

Inductive step $k = 2$ (all states allowed). Set $C = R_{22}^{(1)} = (\varepsilon + b) + a^+ b = b + a^+ b$ (since $\varepsilon + b$ gives ε and b). Then $(R_{22}^{(1)})^* = (b + a^+ b)^*$. Only state 1 is accepting, so $L(\mathcal{A})$ is the set of strings that label paths from 1 to 1:

$$R_{11}^{(2)} = R_{11}^{(1)} + R_{12}^{(1)} (R_{22}^{(1)})^* R_{21}^{(1)} = a^* + a^* b (b + a^+ b)^* a^+.$$

Thus a regular expression R with $L(R) = L(\mathcal{A})$ is

$$\boxed{R = a^* + a^* b (b + a^+ b)^* a^+}.$$

(Here $a^+ = aa^*$; one can also write $a^* b (b \cup a^+ b)^* a^+$ if union is denoted $\cup$.)

Question

If we changed the DFA so that both p and q were accepting states, how would the regular expression R for $L(\mathcal{A})$ change? Would it simplify to a shorter expression?

3 Synthesis

4 Evidence of Participation

5 Conclusion

References

[BLA] Author, [Title](#), Publisher, Year.